JSON. A controller has methods that are mapped to distinct REST methods and request pathways. The *@RequestMapping* annotation, as well as more specific annotations like *@PostMapping*, *@GetMapping*, and others, specify them.

The methods can accept inputs as arguments with annotations indicating how they should be passed, and return responses as output. In a POST request, *@RequestBody* is used to pass a request body. The *@RequestParam* is used to transmit request parameters from the URI as inputs to methods, while *@PathVariable* is used to extract data from the URI. The response type is usually a *ResponseEntity> object* with the domain object's class as its type. A ResponseEntity can be used to return a response code as well as change the request and response body.

Let's take a newsfeed timeline service as an example. The service can add a post to a user's timeline, delete a post from this timeline, show a user's timeline, and fan a post out to other timelines. The controller exposes endpoints for other services to access a user's timeline. The controller may look like this:

```
@GetMapping("/posts")
public ResponseEntity<List<Post>> getPosts() {
    return ResponseEntity.ok(service.getAllPosts());
}
```

In the example, */posts* is the path where the *GET* request is made and it returns a ResponseEntity with *200 OK* status, with the list of all posts available in the service.

To return a specific post with an ID, we can write something like:

```
@GetMapping("/posts/{id}")
public ResponseEntity<List<Post>> getPostById(@PathVariable
String id) {
   return ResponseEntity.ok(service.getPostById(id));
}
```

Here, we're passing a specific ID as a path variable as a parameter to the request. A sample request would be:

```
GET localhost:8080/posts/bP13xxAd9
```

We can also pass request parameters in the URI:

```
   @GetMapping("/posts/filter")
    public ResponseEntity<List<Post>> getPostByFilters(@
RequestParam String tag, @RequestParam String type) {
        return ResponseEntity.ok(service.
getByTagsAndType(tag, type));
    }
```

This will allow us to construct URIs like:

```
GET localhost:8080/posts/filter?tag=sports&type=latest
```

To post something on the timeline service we can use the *@PostMapping* annotation:

```
   @PostMapping("/posts")
    public ResponseEntity<Data> addPost(@RequestBody post) {
        return ResponseEntity.ok(service.
addPostToTimeline(post));
    }
```

Evidently, we're going to avoid writing any business logic in the request response layer and rather take it to the domain layer, which is essentially the service and repository layer.

Let's put all of it together:

```
@RestController
class NewsFeedController {

    private final TimelineService service;

    @Autowired
    public NewsFeedController(TimelineService service) {
        this.service = service;
    }

    @GetMapping("/posts")
    public ResponseEntity<List<Post>> getPosts() {
        return ResponseEntity.ok(service.getAllPosts());
    }

    @GetMapping("/posts/{id}")
    public ResponseEntity<List<Post>> getPostById(@
PathVariable String id) {
        return ResponseEntity.ok(service.getPostById(id));
    }

    @GetMapping("/posts/filter")
    public ResponseEntity<List<Post>> getPostByFilters(@
RequestParam String tag, @RequestParam String type) {
        return ResponseEntity.ok(service.getByTagsAndType(tag,
type));
    }

    @PostMapping("/posts")
    public ResponseEntity<Data> addPost(@RequestBody post) {
        return ResponseEntity.ok(service.
addPostToTimeline(post));
    }
}
```